

# StochLean

## Process Core & Information Flow - Work Handoff Design

Version 0.1

**Primary coverage source: Achim Klenke, Probability Theory: A Comprehensive Course, 3rd ed. (2020)**

Lean baseline for duplicate/API audit: active StochLean pin; initial reference Lean 4 / Mathlib v4.33.0

Scope: stochastic-process core semantics, information flow, random times, and path/measurability interfaces

**WORK HANDOFF SPECIFICATION - IMPLEMENTATION RULES ARE NORMATIVE**

# 1. Purpose, Scope, and Normative Status

**Purpose.** This document is the implementation contract for the StochLean Process Core & Information Flow functional package. It preserves the approved design while removing discussion/history that Work does not need.

- Klenke remains the mathematical coverage/provenance source, but implementation is organized by actual function rather than chapter number.
- This package owns generic process-law/equivalence language, filtration/information-flow interfaces, stopping/random-time semantics, and common path/measurability predicates. It does not own martingale calculus, Kolmogorov-Chentsov regularization, Markov theory, Brownian construction, or path-space weak-convergence topology.
- Coverage does not mean reimplement. Mathlib and approved external code take precedence whenever semantics and usable API already exist.
- Lean files are organized by mathematical function and dependency, never by textbook chapter.
- The only mandatory new code is code that survives the duplicate audit and is required by a BUILD/BUILD-BRIDGE row or an AUDIT-FIRST row resolved to BUILD.
- Examples/exercises are implementation obligations only when explicitly marked load-bearing or acceptance-test material.

| Status          | Meaning                                                                    | Required Work action                                                 |
|-----------------|----------------------------------------------------------------------------|----------------------------------------------------------------------|
| REUSE           | Canonical implementation already exists.                                   | Map to exact declaration/import; do not duplicate.                   |
| REUSE/BRIDGE    | Core exists; thin reusable adapter may be needed.                          | Add only minimal wrapper/specialization with documented API gap.     |
| BUILD           | Confirmed foundation gap.                                                  | Implement rigorously, with no stronger/weaker semantic substitution. |
| BUILD/BRIDGE    | A general result must be added, possibly by composing existing primitives. | Prefer the most generic useful theorem; avoid textbook-only APIs.    |
| BUILD-CANDIDATE | Likely gap, but duplicate audit not yet closed.                            | Search first; implement only if no approved equivalent exists.       |
| AUDIT-FIRST     | Exact Mathlib/approved-external coverage is uncertain.                     | Resolve to REUSE/BRIDGE/BUILD before coding.                         |
| APPLICATION     | Textbook application, not foundation API.                                  | Record coverage; no blocking implementation.                         |
| DEFER           | Valid material outside this milestone.                                     | Record future destination; no current implementation.                |
| EXTERNAL-CHECK  | Textbook cites an external theorem or third-party code may exist.          | Do not trust/import until independent audit is recorded.             |

# 2. Global Non-Negotiable Design Rules

**Coverage rule:** Klenke 3rd edition is the initial mathematical completeness baseline. Do not silently omit a labeled result. If a result is not implemented, its ledger status must explain why.

**Functional organization rule:** No Klenke/ChapterXX.lean layout. Put each object/theorem in one canonical functional module. Cross-chapter material has one implementation only.

**No-duplicate rule:** Before adding any public definition/theorem, search in this exact order: (1) pinned Mathlib revision; (2) the entire local library; (3) the approved-external ledger. An equivalent usable result forbids a parallel implementation.

**No-error rule:** A compiling proof is not sufficient. Do not weaken hypotheses, strengthen conclusions without proof, substitute a surrogate model, hide a conclusion as a structure field, or use vacuous/uninhabited assumptions to close a goal.

**No-semantic-ambiguity rule:** Explicitly distinguish pointwise vs a.e., same one-time law vs f.d.d. law vs coordinate-product law vs topological path-space law, a.s. modification vs indistinguishability, weak convergence vs probability convergence, global vs local convergence in measure, and finite vs extended values.

**No-totalization-abuse rule:** If Mathlib totalizes an operation outside its natural mathematical domain, public theorems must carry the genuine hypotheses or use typed domains. Never exploit fallback values as mathematics.

**Proof-integrity rule:** Project source must contain no sorry, admit, or project-defined axiom. Do not use conclusion-shaped assumptions. Public theorems must pass dependency/axiom inspection.

**Canonical-representation rule:** Use Mathlib-native objects whenever available: Measure/ProbabilityMeasure, Kernel, Lp, Filtration, Adapted, IsProgressive, IsStoppingTime, HasIndepIncrements, etc. Do not create a parallel StochasticProcess structure; processes remain  $X : T \rightarrow \Omega \rightarrow E$ .

**A.s. path rule:** For uncountable time, an a.s. path property means one common full-measure event on which the entire trajectory has the property. Pairwise-a.s. statements are not substitutes.

**Generic-upstream rule:** If a result is not model-specific, place it in a generic namespace/module suitable for future Mathlib upstream. Do not lock generic probability theorems inside Brownian/Poisson/PTASEP namespaces.

**Source-correction rule:** Known textbook errata/semantic defects and explicit source warnings listed in this document override literal OCR/printed shorthand. Work must implement the mathematically correct statement.

**Minimal-import rule:** Final modules should use narrow imports. Avoid import Mathlib in production modules unless no maintainable alternative exists.

### 3. Duplicate, Trust-Boundary, and Provenance Rules

| Layer                        | Trust status                                      | Rule                                                                                                                                                               |
|------------------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lean kernel + pinned Mathlib | Approved baseline                                 | May be reused directly. Exact declaration and import path must be recorded for every REUSE row used by new code.                                                   |
| This StochLean repository    | Approved after CI/audit                           | Search all modules, including the concurrently built Ch.01-08 package, before adding a duplicate. One canonical implementation per semantic object.                |
| Third-party Lean libraries   | Not approved by default                           | BrownianMotion, PFR, StatLean, formal-mathfin, etc. are EXTERNAL-CHECK until a specific module/declaration is audited and entered in an approved-external ledger.  |
| Klenke book                  | Coverage/mathematical source, not code dependency | Use as coverage floor and statement source. Textbook-cited external results remain EXTERNAL-CHECK/DEFER unless separately formalized or reused from approved code. |
| Adapted external code        | Provenance required                               | Record repository, revision, source declaration, license, semantic changes, and local theorem mapping.                                                             |

**HARD GATE: a new public declaration may be merged only after the duplicate audit is recorded. "I did not know the theorem existed" is not an acceptable reason for duplication.**

### 4. Work Execution Protocol, Acceptance, and Longevity

- Step A - Coverage audit: resolve every ledger row and every load-bearing exercise against the pinned Mathlib revision, the complete current local tree, and the approved-external ledger.
- Step B - API design: choose one canonical semantic representation; document natural domain, a.e./version issues, time-index assumptions, and path-vs-coordinate law semantics before proving theorems.
- Step C - Implement only confirmed gaps. BRIDGE code must remain thin; REUSE rows should not spawn local copies.
- Step D - Semantic audit: run the edge cases in Section 9; verify hypotheses/conclusions against Klenke and source warnings.
- Step E - CI gate: lake build; warnings treated as errors for project code; no sorry/admit/project axiom; run lint where applicable; inspect axioms/dependencies of milestone theorems.
- Step F - Coverage gate: every BUILD resolved; every AUDIT-FIRST resolved; every EXTERNAL-CHECK either audited or explicitly deferred with no hidden dependency; exact source mapping recorded.

| Maintenance trigger                                         | Required action                                                                                                                              |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Mathlib version upgrade                                     | Re-run duplicate/API audit for all local public declarations touched by the upgrade; do not carry old REUSE/BUILD decisions forward blindly. |
| New external library approved                               | Re-run overlap search for affected modules; remove/deprecate local duplicates where the approved implementation becomes canonical.           |
| Local generic theorem upstreamed to Mathlib                 | Migrate callers to Mathlib and retire the local copy after compatibility window.                                                             |
| Semantic definition changes                                 | Bump design version and re-run downstream theorem audit; no silent API meaning change.                                                       |
| New Klenke correction/source issue discovered               | Add source-note entry, update ledger and affected formal statement before further dependent work.                                            |
| A theorem requires a project-defined axiom/sorry to proceed | Stop. Mark the item blocked and return it for mathematical/design review; do not weaken the trust boundary.                                  |

### 5. Functional Ownership and Approved Module Layout

Ownership boundary. This package supplies generic semantic glue required before martingale, Markov, Brownian, Levy, stochastic-integration, and process-limit packages. It must stay shallow and reusable.

```

Probability/
Process/
  Equivalence.lean      # only if modification/indistinguishability gap survives audit
  Stationarity.lean     # only if generic process stationarity is absent
Path/
  Continuous.lean       # only missing generic path predicates/bridges
  Cadlag.lean           # predicate layer only; no Skorokhod topology here
Filtration/
  Augmentation.lean     # only if usual-conditions construction is a confirmed gap
Measurability/
  Joint.lean            # only genuine joint/product-measurability bridges
  Progressive.lean      # only genuine progressive/version bridges

```

- This tree is a functional boundary, not a file-creation checklist. If duplicate audit closes a gap, omit the file.

- Do not create local mirrors of Mathlib Process/Filtration.lean, Adapted.lean, Predictable.lean, Stopping.lean, HittingTime.lean, FiniteDimensionalLaws.lean, HasIndepIncrements, or Gaussian-process infrastructure.
- If the Ch.01-08 Work package has already created common-a.s. right-continuous/monotone path predicates or HasStationaryIncrements, reuse them directly and extend only missing generic API.
- Defer Kolmogorov-Chentsov, Doob regularization, full  $D([0, \infty), E)$  topology, martingale calculus, Markov theory, Brownian construction, Levy theory, and stochastic integration to their own functional packages.

## 6. Complete Functional Coverage Ledger - Process Laws and Equivalence

| Source / function                                            | Status                  | Work directive                                                                                                                                   |
|--------------------------------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Klenke Def. 9.6 - process law notation                       | REUSE                   | Use Measure.map / IdentDistrib / Mathlib process-law infrastructure. No new Law structure.                                                       |
| Klenke Def. 9.7(ii) - independent increments                 | REUSE                   | Use Mathlib ProbabilityTheory.HasIndepIncrements.                                                                                                |
| Klenke Def. 9.7(iii) - Gaussian process                      | REUSE                   | Use Mathlib Gaussian-process infrastructure; no second GaussianProcess predicate.                                                                |
| Klenke Def. 9.7(iv) - integrable / square-integrable process | REUSE                   | Prefer forall t, Integrable (X t) P and forall t, MemLp (X t) 2 P; do not add empty wrapper predicates unless later API justifies them.          |
| Klenke Def. 9.7(v) - stationary process                      | AUDIT-FIRST             | If absent, define stationarity by full shifted process law / equivalent f.d.d. shift invariance. Never weaken to equal one-time marginals.       |
| Klenke Def. 9.7(vi) - stationary increments                  | INHERITED / AUDIT-FIRST | Reuse the canonical HasStationaryIncrements from the Ch.01-08 package if built; otherwise close the same previously approved gap exactly once.   |
| Mathlib finite-dimensional process laws                      | REUSE                   | Use FiniteDimensionalLaws.lean for projective f.d.d.s, process coordinate-product law, and modification-to-law bridges.                          |
| Klenke Def. 21.1(i) - modification/version                   | AUDIT-FIRST             | If no canonical predicate exists, BUILD IsModification X Y P := forall t, X t = a.e. Y t.                                                        |
| Klenke Def. 21.1(ii) - indistinguishable                     | AUDIT-FIRST             | If absent, BUILD as one common a.e. event: forall a.e. omega, forall t, X t omega = Y t omega.                                                   |
| Klenke Lemma 21.5(i)                                         | BUILD/BRIDGE if needed  | Countable index: modification implies indistinguishability.                                                                                      |
| Klenke Lemma 21.5(ii)                                        | BUILD/BRIDGE if needed  | Interval index + a.s. right-continuous paths: modification implies indistinguishability. Reuse path predicates and countable dense-set argument. |

- Semantic law hierarchy: one-time law != finite-dimensional law != coordinate-product law != topological path-space law. Mathlib equality of coordinate-product laws from f.d.d.s must not be described as equality of continuous/cadlag path-space laws.
- No generic declaration named EquivalentProcess or SameLaw may collapse modification, indistinguishability, f.d.d. equality, and path-law equality into one ambiguous relation.

## 7. Complete Functional Coverage Ledger - Filtrations and Information Flow

| Source / function                                                                           | Status                       | Work directive                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Klenke Def. 9.9 - filtration                                                                | REUSE                        | Use MeasureTheory.Filtration.                                                                                                                                                     |
| Klenke Def. 9.10 - adapted process                                                          | REUSE                        | Use MeasureTheory.Adapted / StronglyAdapted as appropriate.                                                                                                                       |
| Klenke Remark 9.11 - natural filtration/minimality                                          | REUSE/BRIDGE                 | Use Filtration.natural. Audit exact minimality theorem; add only a thin theorem if missing.                                                                                       |
| Klenke Def. 9.12 - discrete predictable                                                     | REUSE/BRIDGE                 | Use Mathlib predictable sigma-algebra/process API and its discrete characterization. Do not create a second predictable predicate.                                                |
| Klenke Def. 21.22 - right-continuous filtration                                             | REUSE                        | Use Filtration.rightCont and IsRightContinuous.                                                                                                                                   |
| Klenke Def. 21.22 - usual conditions                                                        | AUDIT-FIRST                  | Usual conditions = right continuity + P-completeness at F0. If absent, design a precise property/construction without confusing it with rightCont alone.                          |
| Klenke Remark 21.23 - completed right-continuation                                          | BUILD-CANDIDATE              | Audit Mathlib completion support. A correct augmentation may require changing the ambient measurable space/measurable completion; do not fake a same-ambient Filtration.complete. |
| Klenke Def. 25.5(i) - product measurable                                                    | AUDIT-FIRST                  | Reuse existing joint measurability if present; otherwise only a generic process-level predicate/bridge.                                                                           |
| Klenke Def. 25.5(ii) - progressively measurable                                             | REUSE                        | Use Mathlib IsProgressive / IsStronglyProgressive.                                                                                                                                |
| Klenke Def. 25.5(iii); Thm. 25.8 - predictable continuous-time / predictable -> progressive | REUSE                        | Use Mathlib Filtration.predictable / IsStronglyPredictable and IsStronglyPredictable.isStronglyProgressive; add only genuine measurable-vs-strongly-measurable bridges.           |
| Klenke Remark 25.7 - adapted + product measurable -> progressive modification               | EXTERNAL-CHECK / AUDIT-FIRST | Klenke cites an external source for this converse. Reuse an approved Lean theorem if available; otherwise do not claim textbook proof.                                            |

| Source / function                                                                                   | Status                                   | Work directive                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Klenke Thm. 25.8 - adapted regular paths -> progressive / a.s. regular paths -> progressive version | REUSE/BRIDGE; AUDIT-FIRST (a.s. version) | Reuse Mathlib for adapted right/left-continuous -> progressive, mapping exact hypotheses. For the a.s. right/left-continuous statement, if no exact theorem exists, BUILD a progressively measurable version by modifying on one measurable null event; preserve version semantics explicitly. |

## 8. Complete Functional Coverage Ledger - Random Times and Path Interfaces

| Source / function                                                     | Status                  | Work directive                                                                                                                                                                    |
|-----------------------------------------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Klenke Def. 9.15 - stopping time                                      | REUSE                   | Use $\tau : \Omega \rightarrow \text{WithTop } T$ and <code>MeasureTheory.IsStoppingTime</code> . Do not encode infinity by an arbitrary sentinel value.                          |
| Klenke Thm. 9.16 - countable characterization                         | REUSE                   | Use Mathlib stopping-time equal/level-set API; no duplicate proof unless exact bridge is missing.                                                                                 |
| Klenke Lemma 9.18 - closure operations on stopping times              | REUSE                   | Map min/max/add/shift operations to Mathlib. Preserve hypotheses: look-ahead shifts are not stopping times in general.                                                            |
| Klenke Def. 9.19 - sigma-algebra of tau-past                          | REUSE                   | Use <code>IsStoppingTime.measurableSpace</code> / existing stopped-information sigma algebra.                                                                                     |
| Klenke Lemma 9.21 - $\sigma \leq \tau$ implies $F_\sigma \leq F_\tau$ | REUSE                   | Use existing measurable-space monotonicity if available; thin bridge only if needed.                                                                                              |
| Klenke Def. 9.22 / Lemma 9.23 - $X_\tau$ for countable time           | REUSE                   | Use stopped-value API. Countable-time adaptedness is enough only under the stated hypotheses.                                                                                     |
| Klenke warning after Lemma 9.23                                       | NORMATIVE SEMANTIC RULE | For uncountable time, adaptedness alone does not imply joint measurability or measurability of $X_\tau$ . Any theorem must carry progressive/product/path-regularity assumptions. |
| Klenke Ex. 21.1.4(i)-(iii)                                            | REUSE/BRIDGE            | Right-continuous paths -> joint product measurability; + adapted -> progressive; + stopping time -> $F_\tau$ -measurable evaluation. Use as information-flow acceptance chain.    |
| Mathlib Process/HittingTime.lean                                      | REUSE/AUDIT-FIRST       | Reuse existing hitting-time API. Do not assert arbitrary adapted process + arbitrary Borel target gives a stopping time without the required time/path/topological hypotheses.    |
| Klenke Def. 21.4 - a.s. path properties                               | BUILD/BRIDGE as needed  | All process path predicates use one common full-measure event. Reuse Ch.01-08 right-continuity definitions if present.                                                            |
| Klenke Def. 21.21 - RCLL/cadlag path                                  | BUILD-CANDIDATE         | If absent, add <code>HasCadlagPaths</code> as right-continuous + finite left limit at every $t > 0$ , on one common full-measure event. Predicate layer only.                     |
| Full Skorokhod D-space topology                                       | DEFER                   | Belongs to Functional Weak Convergence & Path-Space Limits package, not Process Core.                                                                                             |
| Klenke Thm. 21.24 - Doob regularization                               | DEFER                   | Belongs to Brownian/Path Regularity or Martingale package after usual-conditions infrastructure is settled.                                                                       |

## 9. Mandatory Semantic Regression Tests and Source Safeguards

- Modification vs indistinguishability: on  $\Omega = T = [0,1]$  with Lebesgue  $P$ ,  $X_t(\omega) = 1_{\{\omega = t\}}$ ,  $Y_t = 0$ . Verify `IsModification` but not `Indistinguishable`.
- Countable-index collapse: for countable  $T$ , modification must imply indistinguishability.
- Right-continuous collapse: for interval  $T$ , two a.s. right-continuous modifications must be indistinguishable (Klenke Lemma 21.5).
- Law hierarchy: the diagonal example may have identical coordinate-product law/f.d.d.s while failing indistinguishability; no theorem may infer pathwise equality from process-law equality.
- Stationarity strength: let  $Z$  be Rademacher and  $X_0 = Z$ ,  $X_1 = Z$ ,  $X_2 = -Z$ . One-time marginals agree, but  $(X_0, X_1)$  and  $(X_1, X_2)$  have different laws. Equal marginals must not satisfy `IsStationary`.
- Natural filtration: verify  $X$  is adapted to `Filtration.natural` and that any filtration to which  $X$  is adapted contains the natural filtration.
- Right continuity vs usual conditions: `rightCont` alone must not imply  $P$ -completeness of  $F_0$ .
- Random-time evaluation: no continuous-time theorem may derive measurability of  $X_\tau$  from Adapted  $X$  alone. Require progressive/joint/path-regularity hypotheses.
- Information-flow chain: adapted + right-continuous -> progressive ->  $X_\tau$  is  $F_\tau$ -measurable, with exact Mathlib/Klenke assumptions.
- Predictability chain: predictable -> progressive -> adapted; discrete predictable characterization agrees with  $F_{\{n-1\}}$ -measurability.
- Infinite stopping time: at least one test uses  $\tau = \text{top}$  on a non-null event and verifies `WithTop` semantics are preserved.
- Cadlag semantics: right continuity and all finite left limits for  $t > 0$  are trajectory-level properties on a single full-measure event.
- Usual augmentation: if implemented, prove the output filtration is right-continuous and completed in the mathematically correct ambient completed space; do not use type-compatible but semantically incomplete shortcuts.

| Source issue / distinction                         | Normative implementation requirement                                                                                                |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Klenke Def. 9.7 stationarity                       | Use shifted full-process law / f.d.d. invariance, not one-time marginal equality.                                                   |
| Klenke Def. 21.1 modification vs indistinguishable | Preserve the two distinct notions exactly; uncountable-time modification does not imply indistinguishability.                       |
| Klenke Lemma 9.23 warning                          | Countable-time proof does not generalize from adaptedness alone; continuous time needs joint/progressive/path regularity.           |
| Klenke Def. 21.22 usual conditions                 | Right continuity and P-completeness are separate requirements.                                                                      |
| Klenke Remark 25.7                                 | The adapted+product-measurable converse is externally cited by Klenke; keep EXTERNAL-CHECK unless independently audited.            |
| Klenke Def. 25.5 terminology                       | Product measurable, progressive, and predictable are distinct properties; do not collapse them.                                     |
| Mathlib Adapted naming change                      | Current Mathlib distinguishes Adapted (measurable) from StronglyAdapted (strongly measurable); map theorem hypotheses deliberately. |

## 10. Work Implementation Order and Milestone Completion Definition

| Phase | Required deliverable                                                                                                                                                                                                        |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A     | Full duplicate/API audit on the pinned Mathlib revision and current local tree, especially outputs of Ch.01-08; resolve every AUDIT-FIRST/BUILD-CANDIDATE row and record exact declarations for REUSE rows used downstream. |
| B     | Process equivalence layer if a real gap survives: modification, indistinguishability, relation API, countable/right-continuous collapse, and bridges to Mathlib finite-dimensional laws.                                    |
| C     | Generic process stationarity only if absent; reuse inherited HasStationaryIncrements. Add law-shift tests that reject marginal-only definitions.                                                                            |
| D     | Path predicate audit/extension: continuous/right/left/cadlag common-a.s. semantics; no D-space topology.                                                                                                                    |
| E     | Filtration audit: natural filtration/minimality, rightCont reuse, then resolve usual-conditions/augmentation gap without ambient-space semantic error.                                                                      |
| F     | Measurability audit: product/progressive/predictable mappings; implement only missing regular-path -> progressive/version bridges.                                                                                          |
| G     | Random-time audit: stopping sigma algebra, stopped value/process, hitting times, and the right-continuous evaluation chain; add only genuine bridges.                                                                       |
| H     | Final coverage, duplicate, semantic, source-warning, dependency/axiom, minimal-import, and CI audit.                                                                                                                        |

**MILESTONE COMPLETE ONLY WHEN: every BUILD is proved; every AUDIT-FIRST and BUILD-CANDIDATE is resolved; every EXTERNAL-CHECK is either audited or explicitly deferred with no hidden dependency; the functional ledger has 100% disposition; and all hard semantic regression tests pass.**

## 11. Expected Net-New Core After Duplicate Audit

- Process modification / indistinguishability predicates and relation bridges, only if current Mathlib/local tree lacks a canonical version.
- Generic process stationarity, only if no canonical implementation exists; stationary increments must reuse the earlier canonical StochLean implementation.
- Common-a.s. cadlag/path predicates only where missing; no duplicate right-continuity predicate from the Ch.01-08 package.
- Usual-filtration augmentation infrastructure only if a mathematically correct gap remains after audit.
- A small number of regular-path -> progressive / stopping-time evaluation bridges only where Mathlib theorem generality or measurable-vs-strongly-measurable API leaves a real gap.

## 12. Mathlib Anchor Modules Already Identified

| Area                                                 | Canonical Mathlib anchors (non-exhaustive)                             |
|------------------------------------------------------|------------------------------------------------------------------------|
| Filtration / natural / right continuation            | Mathlib/Probability/Process/Filtration.lean                            |
| Adapted / progressive measurability                  | Mathlib/Probability/Process/Adapted.lean                               |
| Predictable sigma algebra/processes                  | Mathlib/Probability/Process/Predictable.lean                           |
| Stopping times / stopped values/processes            | Mathlib/Probability/Process/Stopping.lean                              |
| Hitting times                                        | Mathlib/Probability/Process/HittingTime.lean                           |
| Finite-dimensional / coordinate-product process laws | Mathlib/Probability/Process/FiniteDimensionalLaws.lean                 |
| Independent increments                               | Mathlib/Probability/Independence/Process/HasIndepIncrements/Basic.lean |

| Area                                                   | Canonical Mathlib anchors (non-exhaustive)                                                 |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Gaussian processes                                     | Mathlib/Probability/Distributions/Gaussian/IsGaussianProcess/{Def,Basic,Independence}.lean |
| Localizing/stable process properties (future consumer) | Mathlib/Probability/Process/LocalProperty.lean                                             |

Anchor reconnaissance is not a substitute for the required exact duplicate audit at implementation time. The Work report must use the active pinned revision, not silently substitute a newer Mathlib master declaration.

## 13. Source Baseline and Package Boundary

- Achim Klenke, Probability Theory: A Comprehensive Course, Third Edition, Springer, 2020. The user-provided complete PDF is the primary mathematical coverage source for the Klenke items listed here.
- Primary Klenke sources for this package: Section 9.1 (Defs. 9.6-9.12, 9.15-9.23), Section 21.1 (Defs. 21.1, 21.4; Lemma 21.5; Exercise 21.1.4), Section 21.4 (Defs. 21.21-21.22; Remark 21.23), and Section 25.1 (Def. 25.5; Remarks 25.6-25.7; Thm. 25.8).
- Lean/Mathlib baseline: use the same active StochLean pin as the first package; the initial handoff baseline was Lean 4 / Mathlib v4.33.0. Any version change triggers the maintenance rule in Section 4.
- Next functional package after this handoff: Discrete Martingale Calculus. It may consume this package but must not redefine its process-equivalence, filtration, predictable, stopping-time, or path-semantics primitives.